

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Унарные и бинарные операции над графами»

Выполнили:

студенты группы 24ВВВ3

Алмакаев В.В.

Буров А.О.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., Деев М.В.

Пенза 2025

Цель работы – Изучение и практическое освоение унарных и бинарных операций над графами, включая операции, уменьшающие или увеличивающие число элементов графа, такие как удаление и добавление вершин и рёбер, отождествление и расщепление вершин, стягивание рёбер.

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M_1 , M_2 смежности неориентированных помеченных графов G_1 , G_2 . Выведите сгенерированные матрицы на экран.
2. * Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Задание 2

1. Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

2. * Для представления графов в виде списков смежности выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

Задание 3

1. Для матричной формы представления графов выполните операцию:

а) объединения $G = G_1 \cup G_2$

б) пересечения $G = G_1 \cap G_2$

в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции выведите на экран.

Задание 4 *

1. Для матричной формы представления графов выполните операцию декартова произведения графов $G = G_1 \times G_2$.

Результат выполнения операции выведите на экран.

Примечание: задания, помеченные символом * выполняются по указанию преподавателя.

Задание 1.

Код программы

C#

```
using System;

namespace GraphOperations
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            Console.WriteLine("=== ЛАБОРАТОРНАЯ РАБОТА №6 ===");
            Console.WriteLine("Унарные и бинарные операции над графами\n");
```

```
Console.WriteLine("=== ЗАДАНИЕ 1.1 ===");

Console.Write("Введите количество вершин в графах: ");
int vertices = int.Parse(Console.ReadLine());

if (vertices <= 0)
{
    Console.WriteLine("Количество вершин должно быть положительным  
числом.");
    return;
}

int[,] M1 = GenerateAdjacencyMatrix(vertices);
int[,] M2 = GenerateAdjacencyMatrix(vertices);

Console.WriteLine("\nМатрица смежности M1 (граф G1):");
PrintMatrix(M1);

Console.WriteLine("\nМатрица смежности M2 (граф G2):");
PrintMatrix(M2);

Console.WriteLine("\n=== ЗАДАНИЕ 2.1 ===");
Console.WriteLine("Операции с графом G1 (матричная форма):");

int[,] currentGraph = (int[,])M1.Clone();
int currentVertices = vertices;

while (true)
{
    Console.WriteLine("\nТекущий граф:");
    PrintMatrix(currentGraph);
```

```

        Console.WriteLine("\nВыберите операцию:");
        Console.WriteLine("1 - Отождествление вершин");
        Console.WriteLine("2 - Стягивание ребра");
        Console.WriteLine("3 - Расщепление вершины");
        Console.WriteLine("0 - Перейти к следующему заданию");
        Console.Write("Ваш выбор: ");
        int choice = int.Parse(Console.ReadLine());

        if (choice == 0) break;

        switch (choice)
        {
            case 1:
                Console.Write("Введите две вершины для отождествления: ");
                string[] input = Console.ReadLine().Split();
                int v1 = int.Parse(input[0]);
                int v2 = int.Parse(input[1]);
                currentGraph = IdentifyVertices(currentGraph, v1, v2, ref
currentVertices);

                break;

            case 2:
                Console.Write("Введите две вершины для стягивания ребра: ");
                input = Console.ReadLine().Split();
                v1 = int.Parse(input[0]);
                v2 = int.Parse(input[1]);
                currentGraph = ContractEdge(currentGraph, v1, v2, ref
currentVertices);

                break;

            case 3:
                Console.Write("Введите вершину для расщепления: ");

```

```

        v1 = int.Parse(Console.ReadLine());

        currentGraph = SplitVertex(currentGraph, v1, ref
currentVertices);

        break;

    default:

        Console.WriteLine("Неверный выбор!");

        break;

    }

}

```

```

Console.WriteLine("\n=== ЗАДАНИЕ 3 ===");

Console.WriteLine("Бинарные операции над графами G1 и G2:");

```

```

Console.WriteLine("\nИсходный граф G1:");

PrintMatrix(M1);

```

```

Console.WriteLine("\nИсходный граф G2:");

PrintMatrix(M2);

```

```

Console.WriteLine("\n1. Объединение  $G1 \cup G2$ :");

int[,] unionResult = UnionGraphs(M1, M2);

PrintMatrix(unionResult);

```

```

Console.WriteLine("\n2. Пересечение  $G1 \cap G2$ :");

int[,] intersectionResult = IntersectionGraphs(M1, M2);

PrintMatrix(intersectionResult);

```

```

Console.WriteLine("\n3. Кольцевая сумма  $G1 \oplus G2$ :");

int[,] ringSumResult = RingSumGraphs(M1, M2);

PrintMatrix(ringSumResult);

```

```

}

```

```
static int[,] GenerateAdjacencyMatrix(int vertices)
{
    Random rand = new Random();
    int[,] matrix = new int[vertices, vertices];

    for (int i = 0; i < vertices; i++)
    {
        for (int j = i + 1; j < vertices; j++)
        {
            int value = rand.Next(0, 2);
            matrix[i, j] = value;
            matrix[j, i] = value;
        }
    }

    return matrix;
}
```

```
static void PrintMatrix(int[,] matrix)
{
    int size = matrix.GetLength(0);

    Console.Write("    ");
    for (int i = 0; i < size; i++)
        Console.Write($"{i + 1,3}");
    Console.WriteLine();

    for (int i = 0; i < size; i++)
    {
        Console.Write($"{i + 1,2}:");
```

```

        for (int j = 0; j < size; j++)
        {
            Console.Write($"{matrix[i, j],3}");
        }

        Console.WriteLine();
    }
}

```

```

static int[,] IdentifyVertices(int[,] graph, int v1, int v2, ref int vertices)
{
    int size = graph.GetLength(0);
    if (v1 < 1 || v1 > size || v2 < 1 || v2 > size || v1 == v2)
    {
        Console.WriteLine("Ошибка: неверные вершины!");
        return graph;
    }

    int idx1 = v1 - 1;
    int idx2 = v2 - 1;
    int newSize = size - 1;
    int[,] newGraph = new int[newSize, newSize];

    int newRow = 0;
    for (int i = 0; i < size; i++)
    {
        if (i == idx2) continue;

        int newCol = 0;
        for (int j = 0; j < size; j++)

```



```

        {
            if (j == idx2) continue;

            newGraph[newRow, newCol] = graph[i, j];
            newCol++;
        }
        newRow++;
    }

    for (int i = 0; i < newSize; i++)
    {
        if (i == idx1) continue;

        int oldIdx = (i >= idx2) ? i + 1 : i;
        if (graph[oldIdx, idx2] == 1 || newGraph[idx1, i] == 1)
        {
            newGraph[idx1, i] = 1;
            newGraph[i, idx1] = 1;
        }
    }

    vertices = newSize;
    Console.WriteLine($"Вершины {v1} и {v2} отождествлены.");
    return newGraph;
}

static int[,] ContractEdge(int[,] graph, int v1, int v2, ref int vertices)
{
    int idx1 = v1 - 1;
    int idx2 = v2 - 1;

```

```
int size = graph.GetLength(0);

if (v1 < 1 || v1 > size || v2 < 1 || v2 > size || v1 == v2)
{
    Console.WriteLine("Ошибка: неверные вершины!");
    return graph;
}

if (graph[idx1, idx2] == 0)
{
    Console.WriteLine($"Ошибка: ребро между вершинами {v1} и {v2} не
существует!");
    return graph;
}

int newSize = size - 1;
int[,] newGraph = new int[newSize, newSize];

int newRow = 0;
for (int i = 0; i < size; i++)
{
    if (i == idx2) continue;

    int newCol = 0;
    for (int j = 0; j < size; j++)
    {
        if (j == idx2) continue;

        newGraph[newRow, newCol] = graph[i, j];
        newCol++;
    }
}
```

```

        newRow++;
    }

    for (int i = 0; i < newSize; i++)
    {
        if (i == idx1) continue;

        int oldIdx = (i >= idx2) ? i + 1 : i;
        if (graph[oldIdx, idx2] == 1 || newGraph[idx1, i] == 1)
        {

            newGraph[idx1, i] = 1;
            newGraph[i, idx1] = 1;
        }
    }

    vertices = newSize;
    Console.WriteLine($"Ребро между вершинами {v1} и {v2} стянуто.");
    return newGraph;
}

static int[,] SplitVertex(int[,] graph, int v, ref int vertices)
{
    int idx = v - 1;
    int size = graph.GetLength(0);

    if (v < 1 || v > size)
    {
        Console.WriteLine("Ошибка: неверная вершина!");
        return graph;
    }

```

```

    }

    int newSize = size + 1;
    int[,] newGraph = new int[newSize, newSize];

    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            newGraph[i, j] = graph[i, j];
        }
    }

    newGraph[idx, newSize - 1] = 1;
    newGraph[newSize - 1, idx] = 1;

    for (int i = 0; i < size; i++)
    {
        if (graph[idx, i] == 1)
        {
            newGraph[newSize - 1, i] = 1;
            newGraph[i, newSize - 1] = 1;
        }
    }

    vertices = newSize;
    Console.WriteLine($"Вершина {v} расщеплена на вершины {v} и {newSize}.");
    return newGraph;
}

```

```
static int[,] UnionGraphs(int[,] G1, int[,] G2)
{
    int size1 = G1.GetLength(0);
    int size2 = G2.GetLength(0);
    int maxSize = Math.Max(size1, size2);

    int[,] result = new int[maxSize, maxSize];

    for (int i = 0; i < maxSize; i++)
    {
        for (int j = 0; j < maxSize; j++)
        {
            int val1 = (i < size1 && j < size1) ? G1[i, j] : 0;
            int val2 = (i < size2 && j < size2) ? G2[i, j] : 0;
            result[i, j] = (val1 == 1 || val2 == 1) ? 1 : 0;
        }
    }

    return result;
}
```

```
static int[,] IntersectionGraphs(int[,] G1, int[,] G2)
{
    int size1 = G1.GetLength(0);
    int size2 = G2.GetLength(0);
    int minSize = Math.Min(size1, size2);

    int[,] result = new int[minSize, minSize];

    for (int i = 0; i < minSize; i++)
```

```

    {
        for (int j = 0; j < minSize; j++)
        {
            result[i, j] = (G1[i, j] == 1 && G2[i, j] == 1) ? 1 : 0;
        }
    }

    return result;
}

```

```

static int[,] RingSumGraphs(int[,] G1, int[,] G2)
{
    int size1 = G1.GetLength(0);
    int size2 = G2.GetLength(0);
    int maxSize = Math.Max(size1, size2);

    int[,] result = new int[maxSize, maxSize];

    for (int i = 0; i < maxSize; i++)
    {
        for (int j = 0; j < maxSize; j++)
        {
            int val1 = (i < size1 && j < size1) ? G1[i, j] : 0;
            int val2 = (i < size2 && j < size2) ? G2[i, j] : 0;
            result[i, j] = (val1 != val2) ? 1 : 0;
        }
    }

    for (int i = 0; i < maxSize; i++)
    {
        result[i, i] = 0;
    }
}

```

```

    }

    return result;
}

}

}

```

Результаты работы программ

```

=== ЛАБОРАТОРНАЯ РАБОТА №6 ===
Унарные и бинарные операции над графами

=== ЗАДАНИЕ 1.1 ===
Введите количество вершин в графах: 5

Матрица смежности M1 (граф G1):
    1  2  3  4  5
1:  0  1  1  1  1
2:  1  0  0  1  1
3:  1  0  0  0  0
4:  1  1  0  0  0
5:  1  1  0  0  0

Матрица смежности M2 (граф G2):
    1  2  3  4  5
1:  0  1  0  0  0
2:  1  0  0  0  0
3:  0  0  0  0  1
4:  0  0  0  0  0
5:  0  0  1  0  0

=== ЗАДАНИЕ 2.1 ===
Операции с графом G1 (матричная форма):

Текущий граф:
    1  2  3  4  5
1:  0  1  1  1  1
2:  1  0  0  1  1
3:  1  0  0  0  0
4:  1  1  0  0  0
5:  1  1  0  0  0

Выберите операцию:
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Перейти к следующему заданию
Ваш выбор: 1
Введите две вершины для отождествления: 2 5
Вершины 2 и 5 отождествлены.

Текущий граф:
    1  2  3  4
1:  0  1  1  1
2:  1  0  0  1
3:  1  0  0  0
4:  1  1  0  0

```

Рисунок 1 - Результат работы программы 1

Текущий граф:

	1	2	3	4
1:	0	1	1	1
2:	1	0	0	1
3:	1	0	0	0
4:	1	1	0	0

Выберите операцию:

- 1 - Отождествление вершин
- 2 - Стягивание ребра
- 3 - Расщепление вершины
- 0 - Перейти к следующему заданию

Ваш выбор: 2

Введите две вершины для стягивания ребра: 2 4

Ребро между вершинами 2 и 4 стянуто.

Текущий граф:

	1	2	3
1:	0	1	1
2:	1	0	0
3:	1	0	0

Рисунок 2 - Результат работы программы 2


```
=== ЗАДАНИЕ 3 ===  
Бинарные операции над графами G1 и G2:
```

```
Исходный граф G1:
```

	1	2	3	4	5
1:	0	1	1	1	1
2:	1	0	0	1	1
3:	1	0	0	0	0
4:	1	1	0	0	0
5:	1	1	0	0	0

```
Исходный граф G2:
```

	1	2	3	4	5
1:	0	1	0	0	0
2:	1	0	0	0	0
3:	0	0	0	0	1
4:	0	0	0	0	0
5:	0	0	1	0	0

```
1. Объединение G1 ∨ G2:
```

	1	2	3	4	5
1:	0	1	1	1	1
2:	1	0	0	1	1
3:	1	0	0	0	1
4:	1	1	0	0	0
5:	1	1	1	0	0

```
2. Пересечение G1 ∧ G2:
```

	1	2	3	4	5
1:	0	1	0	0	0
2:	1	0	0	0	0
3:	0	0	0	0	0
4:	0	0	0	0	0
5:	0	0	0	0	0

```
3. Кольцевая сумма G1 ⊕ G2:
```

	1	2	3	4	5
1:	0	0	1	1	1
2:	0	0	0	1	1
3:	1	0	0	0	1
4:	1	1	0	0	0
5:	1	1	1	0	0

Рисунок 3 - Результат работы программы 3

Вывод: В ходе выполнения лабораторной работы была разработана программа на языке C#, реализующая основные унарные и бинарные операции над графами.